



**TECNOLÓGICO
DE MONTERREY®**

Instituto Tecnológico y de Estudios Superiores de Monterrey.
Ingeniería en Tecnologías Computacionales.

ANALIZADOR SINTÁCTICO.

DOCENTE: RODOLFO JULIO CASTELLO ZETINA.

DESARROLLO DE APLICACIONES AVANZADAS DE CIENCIAS
COMPUTACIONALES

Alumna:

Campos Rivera Ana Victoria.

Matrícula: A01567111

Fecha de entrega: 11/06/2026

1. Introducción

1.1. Resumen

El objetivo de este proyecto fue desarrollar un analizador sintáctico (parser) del lenguaje C++. Este analizador corresponde a la segunda fase del proceso de traducción de un compilador. Previo a continuar con este reporte, es significativo explicar el flujo del funcionamiento del parser. Este inicia recibiendo un flujo de tokens generado por el analizador léxico, para posteriormente verificar que su secuencia respete la estructura gramatical del lenguaje, construyendo así, un árbol sintáctico representativo de dicha estructura [1]. En esta implementación, además de reconocer la estructura sintáctica y reportar errores, se encarga de construir y actualizar una tabla de símbolos durante el parseo.

El analizador se desarrolló haciendo uso de varias herramientas. Para el scanner, se hizo uso de Lex, para el parser se hizo uso de Yacc, y para el código de las acciones y las estructuras de datos, se utilizó el lenguaje C. Cabe destacar que, debido a las limitaciones impuestas en el documento de instrucciones, no se hizo uso de librerías ni de APIs de gramáticas libres de contexto.

Este reporte es la documentación del proceso de desarrollo. Se incluye el análisis de requerimientos sintácticos, corrección de la gramática del borrador, el diseño de la integración entre el analizador léxico y esta nueva fase, la implementación del archivo Yacc y también la validación por medio de casos de prueba.

1.2. Notación

- **Gramática libre de contexto:** La gramática libre de contexto se define como una cuádrupla $G = (V, T, P, S)$ en donde V corresponde a un conjunto de no terminales, T a un conjunto de terminales, P a un conjunto de producciones y S al símbolo inicial [1], [2]. Las gramáticas que se diseñan para Yacc, se describen en notación de Backus Nour Form [3].

Varios de estos términos pueden parecer extraños o muy técnicos, estos se explican mejor en la tabla a continuación:

Símbolo	Significado
→	"se deriva en" o "produce". Se encarga de separar el no terminal (izquierda) de su definición (derecha). En Yacc se escribe con ::
ϵ	Este símbolo representa una cadena vacía. $A \rightarrow \epsilon$ indicaría que A puede no generar ningún símbolo.
Terminal	Una terminal corresponde a un token del lenguaje (ej., ID, IF, SEMICOLON).
No terminal	Es una variable gramatical y se escribe en minúsculas (p. ej., statement, expression).

Tabla 1. Definiciones gramática libre de contexto

- **Gramática ambigua:** Una gramática es considerada ambigua cuando una cadena admite dos árboles de derivación diferentes. Esto representa un problema ya que el parser no puede determinar con precisión la estructura del programa [1].
- **Modelo de desarrollo:** Para crear el analizador se siguió el proceso clásico de software: análisis, diseño, implementación y pruebas conforme al IEE-830 [4]. Se decidió trabajar de esta forma ya que crear un compilador es un proceso en el que cada fase transforma el programa y su calidad depende directamente de la correctitud de la gramática obtenida durante el análisis.
- **Justificación lenguaje y herramientas:** El parser se hizo con Yacc y el scanner se hizo con Lex. Se utilizaron estas herramientas ya que Yacc construye automáticamente el parser y permite conservar la recursión izquierda natural del lenguaje y adicionalmente, la integración Lex-Yacc por medio del archivo y.tab.h crea una conexión directa entre los tokens del scanner y el parser [3].

2. Análisis

El propósito de esta sección es transformar la descripción informal del lenguaje, en una gramática correcta, completa y no ambigua que se pueda implementar en Yacc.

2.1. Análisis de los requerimientos sintácticos

Las construcciones que el parser debe reconocer son las siguientes:

- **Declaraciones de variables:** Estas pueden ser simples o pueden ser un arreglo. Los tipos son int, float o string.
- **Declaraciones de funciones:** Se debe reconocer el tipo de retorno, el identificador, los parámetros y el cuerpo. Un programa consiste en una secuencia de declaraciones.
- **Parámetros:** Son por valor, por referencia (arreglos), o void.
- **Bloques:** También conocidos como compound statements, estos son. Las llaves que encierran las declaraciones locales y una lista de sentencias.
- **Asignaciones:** Se deben reconocer las asignaciones de una expresión o de una constante de cadena a una variable.
- **Condicionales:** Se debe reconocer un if y su else que puede ser opcional.
- **Ciclos:** While.
- **Retornos:** Return que puede no tener valor.
- **Lectura y escritura:** El read de una variable y el write de una expresión.
- **Expresiones:** Relacionales (un solo operador relacional) y aritméticas (+ - * /).
- **Llamadas a funciones:** Identificador con sus argumentos entre paréntesis.
- **Arreglos:** Una declaración con tamaño entero y acceso por subíndice.

2.2. Revisión de la gramática proporcionada

Se revisaron las 31 producciones proporcionadas en el documento de instrucciones y se detectaron los siguientes problemas:

- **Producción rota (regla 14):** En las instrucciones se define `assignment_stmt → var = expression ; | STRING`. Esto no permite representar correctamente una asignación de cadena ya que no se puede asignar una cadena literal sin variable ni `=`. La forma correcta sería `assignment_stmt → var = expression ; | var = STRING ;`

- **Terminal NUMBER único:** En las instrucciones se usa únicamente el terminal NUMBER pero en el scanner ya se habían identificado dos tokens: INTEGER y REAL.
- **Ambigüedad (dangling-else):** El else opcional puede relacionarse con más de un if. Por ejemplo, en `if (e1) if (e2) s1 else s2`, no queda claro si el else pertenece al primer if o al segundo. Por eso, la expresión puede generar dos interpretaciones distintas.
- **Inconsistencia de nomenclatura:** Existe inconsistencia en las reglas 30 y 31. En una se usa `arg_list` y en la otra `args_list`. Ambos nombres se refieren a lo mismo y se debe utilizar un solo nombre en la gramática para evitar errores.
- **Restricciones semánticas:** Algunas reglas no se pueden controlar únicamente con la gramática. Un ejemplo es la regla 5 que permite usar `void` como un tipo de variable aunque en las instrucciones se indica que esto no está permitido. Ocurre lo mismo con verificar que una variable haya sido declarada antes de usarse. Todas estas condiciones se deben revisar durante el análisis semántico.
- **Signo incluido en los números:** En la fase de análisis léxico, los números aceptaban un signo opcional. Esto causa problemas en el parser ya que una expresión como `5-3` podría reconocerse como un `5` y un `-3` en lugar de como una operación de resta. Por esto, los signos `+` y `-` se deben reconocer como tokens separados.

2.3. Decisiones y modificaciones realizadas

Todos los problemas mencionados anteriormente fueron solucionados. Para cada cambio, se registra la producción original y la modificación correcta. Estas se enlistan a continuación:

- **Producción rota (regla 14):**

Original: `assignment_stmt → var = expression ; | STRING ;`

Modificación: `assignment_stmt → var = expression ; | var = STRING_LITERAL ;`

- **Terminal NUMBER único:**

Original: `var_declaration → ... ID [ NUMBER ] ;` y `factor → ... | NUMBER.`

Modificación: `[ INTEGER ]` para el tamaño del arreglo y `factor → ... | INTEGER | REAL.`

- **Ambigüedad (dangling-else):**

La regla original permite dos formas de `if`, una con `else` y otra sin `else`, lo que puede causar ambigüedad. Para resolverlo, se mantienen las

dos producciones, pero como solución, en Yacc se asigna menor precedencia al if sin else mediante el token ficticio `LOWER_THAN_ELSE`. Después, `ELSE` recibe una precedencia mayor. De esta forma, el else queda relacionado con el if más cercano.

- **Inconsistencia de nomenclatura:**

Original: `nombres` `arg_list` y `args_list` usados indistintamente.

Modificación: nomenclatura unificada a `args` / `arg_list`.

- **Restricciones semánticas:**

Habían condiciones mezcladas con las reglas sintácticas, aunque no pueden validarse únicamente con la gramática. Por ejemplo, no se permite declarar variables de tipo void y las variables deben estar declaradas antes de utilizarse. Estas restricciones se revisan con las acciones semánticas del parser y la tabla de símbolos. Esto es correcto porque dependen no solo de la estructura del programa sino de su contexto.

- **Signo incluido en los números:**

Original: `INTEGER` $\rightarrow$ `[+-]?{DIGIT}+` y `REAL` $\rightarrow$ `[+-]?{DIGIT}+.{DIGIT}+` (signo incluido en el token).

Modificación: `INTEGER` $\rightarrow$ `{DIGIT}+`, `REAL` $\rightarrow$ `{DIGIT}+.{DIGIT}+`

2.4. Simplificaciones de la gramática

Se revisaron todos los tipos de simplificación que se vieron en clase. En algunos casos se decidió conservar las producciones y en otros sí simplificar. A continuación se justifican las decisiones tomadas:

- **Símbolos inútiles:** Un símbolo se considera inútil si este no puede derivar un terminal [1]. Se verificó que todos los no terminal de la gramática derivaran en terminales.
- **Símbolos inalcanzables:** Un símbolo es inalcanzable cuando no se puede llegar a él desde un símbolo inicial [1]. Después de unificar `args` y `arg_list`, todos los no terminales son alcanzables desde `program`.
- **Producciones epsilon:** Se conservan tres de estas producciones: `local_declaratuons`, `statement_list` y `args`. Estas son necesarias para poder expresar cuerpos y listas vacías. Un parser ascendente las puede manejar sin problema y eliminarlas sería complicar la gramática.

- **Producciones unitarias:** Las producciones unitarias se conservan porque estas son estructurales. Es decir, definen los puntos donde se enganchan las acciones semánticas y la jerarquía de precedencia. Eliminar estas producciones sería eliminar la estructura que el parser necesita [1].
- **Producciones redundantes:** No se encontraron producciones que generen el mismo lenguaje de forma redundante.
- **Recursión:** Se mantiene la recursión izquierda. En un parser descendente esto crearía ciclos infinitos pero con Yacc se trata de un parser ascendente y este lo puede manejar.
- **Precedencia y asociatividad:** La estructura `expression → arithmetic_expression → term → factor` permite que la multiplicación y la división se resuelvan antes que la suma y la resta. También hace que las operaciones se agrupen de izquierda a derecha. Por eso no fue necesario usar declaraciones `%left` para los operadores aritméticos. El único conflicto que se tuvo que resolver por separado fue el del dangling else.

2.5. Análisis semántico

Durante el parseo, el parser se encarga de construir una tabla de símbolos y de incorporar la siguiente información y validaciones:

- **Variable declarada:** Para cada uso de una variable, se verifica que esta ya esté en la tabla. Si no está, se reporta un error.
- **Función declarada antes de llamarse:** Para cada llamada de una función se verifica que esta ya exista en la tabla. Las funciones recursivas sí funcionan porque se registran en la tabla antes de parsear su cuerpo.
- **Tipos de variable y funciones:** Se registra el tipo que corresponde a cada identificador. Estos tipos pueden ser `int`, `float`, `string` y `void`.
- **Parámetros y arreglos:** Los parámetros se registran en ámbito local de su función y se marca si un identificador es un arreglo.
- **Alcance de identificadores:** Se distingue entre el ámbito local y global por función. Si una variable local tiene el mismo nombre que una global, se utiliza la variable local dentro de esa función.

- **Redeclaración:** Se reporta un error cuando un identificador se declara más de una vez dentro del mismo ámbito.
- **Restricción de variables void:** No se permite declarar variables de tipo void. El parser indica un error.

Hay validaciones que aún podrían aplicarse pero estas corresponderían a una fase de análisis semántico más completa. Algunos ejemplos serían la validación de la cantidad y el tipo de argumentos en una función y el uso correcto de arreglos.

Esta implementación se limitó a las validaciones que se vieron en clase. Principalmente el registro de los identificadores en una tabla de símbolos y la comprobación de que se hayan declarado antes de usarse.

2.6. Gramática final

En la siguiente tabla se presenta la gramática libre de contexto final que se usó en la implementación en Yacc. Las terminales están en mayúsculas.

#	No terminal	Producción	Explicación
1	program	declaration_list	Un programa es una secuencia de declaraciones.
2	declaration_list	declaration_list declaration	Una o más declaraciones (recursión izquierda).
3	declaration	var_declaration fun_declaration	Una declaración puede ser de variable o de función.
4	var_declaration	type_specifier ID ; type_specifier ID [INTEGER] ;	Variable simple o arreglo de tamaño entero.

5	type_specifier	INT FLOAT STRING VOID	Tipos básicos. El tipo VOID solo es válido en funciones.
6	fun_declaration	type_specifier ID (params) compound_stmt	Función: tipo, nombre, parámetros y cuerpo.
7	params	param_list VOID	Lista de parámetros o sin parámetros.
8	param_list	param_list , param param	Uno o más parámetros separados por una coma.
9	param	type_specifier ID type_specifier ID []	Parámetro normal o arreglo.
10	compound_stmt	{ local_declarations statement_list }	Bloque con declaraciones locales y sentencias.
11	local_declarations	local_declarations var_declaration ϵ	Cero o más declaraciones locales.
12	statement_list	statement_list statement ϵ	Cero o más sentencias.
13	statement	assignment_stmt call_stmt compound_stmt selection_stmt iteration_stmt return_stmt input_stmt output_stmt	Tipos de sentencia.

14	assignment_stmt	<code>var = expression ; var = STRING_LITERAL ;</code>	Asignación de expresión o de cadena.
15	call_stmt	<code>call ;</code>	Llamada a función como sentencia.
16	selection_stmt	<code>IF (expression) statement [%prec LOWER_THAN_ELSE] IF (expression) statement ELSE statement</code>	Condicional.
17	iteration_stmt	<code>WHILE (expression) statement</code>	Ciclo.
18	return_stmt	<code>RETURN ; RETURN expression ;</code>	Retorno con o sin valor.
19	input_stmt	<code>READ var ;</code>	Lectura a una variable.
20	output_stmt	<code>WRITE expression ;</code>	Escritura de una expresión.
21	var	<code>ID ID [arithmetic_expression]</code>	Variable simple o acceso a arreglo.
22	expression	<code>arithmetic_expression relop arithmetic_expression arithmetic_expression</code>	A lo sumo un operador relacional (sin asociar).
23	relop	<code><= < > >= == !=</code>	Operadores relacionales.

24	arithmetic_expression	arithmetic_expression addop term term	Suma/resta, asociatividad izquierda.
25	addop	+ -	Operadores aditivos.
26	term	term mulop factor factor	Multiplicación/división, mayor precedencia.
27	mulop	* /	Operadores multiplicativos.
28	factor	(arithmetic_expression) var call INTEGER REAL	Factor: paréntesis, variable, llamada o número.
29	call	ID (args)	Llamada con argumentos.
30	args	arg_list ϵ	Argumentos o vacío.
31	arg_list	arg_list arithmetic_expression arithmetic_expression	Lista de argumentos.

Tabla 2. Gramática libre de contexto utilizada

3. Diseño

En esta sección se explica cómo se implementó lo definido durante la fase de análisis.

3.1. Organización general del parser

El analizador se organiza en dos archivos fuentes y cada uno tiene una responsabilidad diferente. Adicionalmente, se incluye un Makefile de construcción.

- o **src/scanner.l**: Este se encarga de reconocer los tokens del archivo fuente y se los entrega al parser.
- o **src/parser.y**: Tiene la gramática, las acciones semánticas, tabla de símbolos y el main().

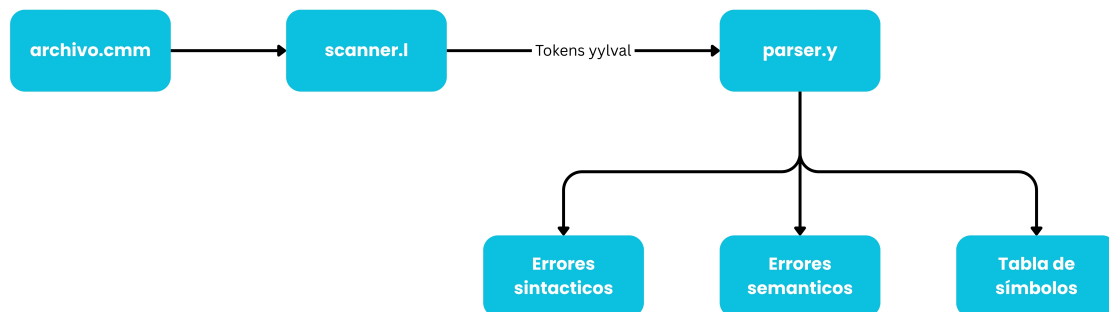


Figura 1. Flujo de información desde el archivo fuente hasta salida

El parser invoca repetidamente a `yylex()` cada vez que necesita un nuevo token. El scanner reconoce el siguiente lexema, devuelve el token y cuando es necesario guarda el valor en `yylval`. Cuando termina el análisis, muestra el resultado y la tabla de símbolos.

3.2. Organización general del parser

La conexión entre Lex y Yacc se hace por medio del archivo `y.tab.h` que genera yacc con la opción `-d` [3].

- El parser declara todos los terminales con `%token`. Cuando compila con bison, se genera `y.tab.h` que incluye un `#define` por cada token y su definición correspondiente de tipo `YYSTYPE`.
- El scanner incluye `y.tab.h` y por cada lexema que reconoce hace un `return TOKEN`, devolviendo el código que el parser espera.
- La información extra se comparte con variables globales. El lexema viaja en `yyval` y el número de línea en `yylineno`.
- `yylex()` entrega un token a la vez. El parser lo pide, el scanner lo reconoce, asigna `yyval` si el token transporta un valor, y retorna el código.

3.3. Actualización de la especificación de Lex

En esta fase, el scanner cambió de rol. Antes se encargaba de imprimir tokens y construía cinco tablas. Ahora es un proveedor de tokens para el parser. Los cambios específicos son los siguientes:

- **Tokens:** Los tokens cambiaron en representación. Ahora son las constantes que define el parser con `%token` en lugar de los `#define` numéricos locales.
- **Expresiones regulares:** Se conservan las definiciones que se aprobaron en la fase de análisis léxico. Los ajustes que se hicieron fue que los números ahora ya no incluyen signo para que este pueda ser reconocido como operador y los comentarios ahora se reconocen con una expresión regular.
- **Palabras reservadas, operadores, y símbolos especiales:** Cada uno hace `return` del token que le corresponde. Los de dos caracteres, van antes que los de uno.
- **Eliminación del main:** El `main` ahora lo provee el parser.
- **Conteo de errores léxicos:** Cada error incrementa la variable global `lexical_errors`. Cuando se termina el análisis, `main()` revisa la variable para evitar que un archivo con errores se marque como correcto.

- **Números reales con sufijos inválidos:** Se añadió la regla `INVALID_REAL_SUFFIX` que detecta entradas como `12.5abc` como un solo número inválido en lugar de como un número y un identificador.
- **%option nodefault:** Esta opción evita que Flex aplique su acción predeterminada.

A continuación se presenta la nueva versión del lexer:

```

INVALID_REAL_NO_INT    \.{DIGIT}+
INVALID_REAL_SUFFIX    {DIGIT}+\.{DIGIT}+({LETTER}|_){LETTER}/{DIGIT}|_)*
INVALID_ID_UNDERSCORE  (_|_){LETTER}/{DIGIT}|_)*

%%
{BLANK}+                { /* se ignoran los espacios en blanco */ }
{COMMENT}               { /* se ignoran los comentarios bien formados */ }
}
"/*"                   { int c;
                        fprintf(stderr, "Lexical error at line %d:
unclosed comment\n", yylineno);
                        lexical_errors++;
                        while ((c = input()) != EOF) { /* consume el
resto del archivo */ } }

{INVALID_ID_DIGIT}      { fprintf(stderr, "Lexical error at line %d:
invalid identifier '%s'\n", yylineno, yytext); lexical_errors++; }
{INVALID_REAL_SUFFIX}   { fprintf(stderr, "Lexical error at line %d:
invalid real number '%s'\n", yylineno, yytext); lexical_errors++; }
{INVALID_REAL_TWO_DOTS} { fprintf(stderr, "Lexical error at line %d:
invalid real number '%s'\n", yylineno, yytext); lexical_errors++; }
{INVALID_REAL_END_DOT}  { fprintf(stderr, "Lexical error at line %d:
invalid real number '%s'\n", yylineno, yytext); lexical_errors++; }
{INVALID_REAL_NO_INT}   { fprintf(stderr, "Lexical error at line %d:
invalid real number '%s'\n", yylineno, yytext); lexical_errors++; }
{INVALID_ID_UNDERSCORE} { fprintf(stderr, "Lexical error at line %d:
invalid identifier '%s'\n", yylineno, yytext); lexical_errors++; }
\"[^\\n\\"]*           { fprintf(stderr, "Lexical error at line %d:
unclosed string '%s'\n", yylineno, yytext); lexical_errors++; }

/* Palabras reservadas (insensibles a mayusculas/minusculas) */
[Ii][Nn][Tt]           { return INT;      }
[Ff][Ll][Oo][Aa][Tt]   { return FLOAT;    }
[Ss][Tt][Rr][Ii][Nn][Gg] { return STRING; }
[Rr][Ee][Tt][Uu][Rr][Nn] { return RETURN; }
[Ii][Ff]               { return IF;        }
[Ee][Ll][Ss][Ee]       { return ELSE;     }
[Ww][Hh][Ii][Ll][Ee]   { return WHILE;    }
[Rr][Ee][Aa][Dd]       { return READ;     }
[Ww][Rr][Ii][Tt][Ee]   { return WRITE;    }
[Vv][Oo][Ii][Dd]       { return VOID;     }

/* Tokens con valor: numeros, cadenas e identificadores */
{REAL}                 { yylval.dval = atof(yytext); return REAL; }
{INTEGER}              { yylval.ival = atoi(yytext); return INTEGER; }
{STRING_LITERAL}       { yylval.str  = strdup(yytext); return
STRING_LITERAL; }
{ID}                   { yylval.str  = strdup(yytext); return ID; }

/* Operadores (Los de dos caracteres van primero) */

```

```

"<="      { return LTE; }
">="      { return GTE; }
"=="      { return EQ; }
"!="      { return NEQ; }
"!"       { fprintf(stderr, "Lexical error at line %d:
invalid operator '%s'\n", yylineno, yytext); lexical_errors++; }
"<"       { return LT; }
">"       { return GT; }
"="       { return ASSIGN; }
"+"       { return PLUS; }
"-"       { return MINUS; }
"*"       { return TIMES; }
"/"       { return DIVIDE; }

/* Delimitadores */
";"       { return SEMICOLON; }
","       { return COMMA; }
"("       { return LPAREN; }
")"       { return RPAREN; }
"["       { return LBRACKET; }
"]"       { return RBRACKET; }
"{"       { return LBRACE; }
"}"       { return RBRACE; }

.         { fprintf(stderr, "Lexical error at line %d:
invalid symbol '%s'\n", yylineno, yytext); lexical_errors++; }

%%

```

3.4. Extensión de las tablas de símbolos

El parser usa una tabla de símbolos para almacenar la información de las variables y funciones declaradas. Cada entrada de la estructura SymEntry contiene los siguientes campos:

Campo	Descripción
name	Nombre del identificador.
type	Tipo de dato: int, float, string o void.
kind	Variable o función.
isArray	Indica si es arreglo (1) o escalar (0).
scope	Alcance: 0 = global; n = ámbito local de la n-ésima función.
line	Línea de declaración.

Tabla 3. Campos de estructura SymEntry

Se implementaron dos niveles de alcance:

- Ámbito global para las funciones y para las variables globales.
- Ámbito local para los parámetros y las variables de cada función.

Cada vez que inicia el análisis de una función, se crea un identificador de ámbito. Todos sus parámetros y variables quedan registrados dentro de este ámbito. Cuando termina la función, el parser regresa al ámbito global.

Las entradas no se eliminan de la tabla al terminar la función, estas únicamente dejan de estar visibles para búsquedas posteriores. Esto se realiza de esta forma para poder imprimir todos los símbolos encontrados en el análisis al final.

En los casos en los que existe una variable local con el mismo nombre que una global, se da prioridad a la declaración local.

La implementación actual usa un ámbito local para toda la función. Los bloques definidos con `{ }` no crean un ámbito nuevo. Manejar ámbitos anidados sería parte de una fase más avanzada.

3.5. Manejo y vinculación de tokens

Adentro del archivo Yacc, los terminales son declarados con `%token` y los que transportan valor indican el campo `%union` que usan. Cuando se genera `y.tab.h` las declaraciones quedan disponibles para el scanner y así los dos archivos usan los mismos tokens y la misma estructura `YYSTYPE`.

En las reglas de Yacc:

- `$1`, `$2`, `$3`, representan los valores de los símbolos de la producción.
- `$$` es el valor que produce el no terminal del lado izquierdo.
- `yylval` transporta el valor del scanner al parser.

La siguiente tabla muestra la relación que existe entre los requerimientos, las reglas gramaticales y su implementación

Requerimiento	Producción o regla	Regla en Yacc	Implementación
Declaración de variables	<code>var_declarati on</code>	<code>var_declaration,</code> <code>local_declarations</code>	<code>declareVar()</code> en <code>parser.y</code>
Declaración de funciones	<code>fun_declarati on</code>	Acción intermedia de <code>fun_declaration</code>	<code>startFunction()</code> y <code>endFunction()</code>

Parámetros	params, param_list, param	Reglas de parámetros	declareVar()
Asignación	assignment_stmt	Asignación de expresión o cadena	Reglas de parser.y
Condicional	selection_stmt	Uso de %prec LOWER_THAN_ELSE	Declaraciones de precedencia
Variable declarada antes de usarse	var	Regla var	useVar()
Función declarada antes de llamarse	call	Regla call	useFunc()
Precedencia de expresiones	expression, term, factor	Jerarquía de reglas	Estructura de la gramática
Reconocimiento de tokens	Terminales	%token y return	scanner.l y y.tab.h
Errores sintácticos	declaration y statement	error SEMICOLON	yyerror() y yyerrok

Tabla 3. Relación entre reglas, reglas gramaticales y su implementación

4. Implementación con Yacc

Esta sección se adentra en Yacc y el archivo parser.y. Este archivo se organiza en tres secciones principales: sección de definiciones, sección de reglas gramaticales y sección de código de usuario. Todas las secciones se separan con %%.

4.1. Sección de definiciones

Esta es la primera sección del archivo. Contiene, entre %{ y %}, el código en C que se coloca al inicio del parser generado. En esta sección se incluyen las bibliotecas usadas:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Adicionalmente se declaran las funciones y variables que proporciona el scanner:

- o yylex(): entrega al parser el siguiente token reconocido.
- o yylineno: contiene el número de línea actual.
- o yytext: contiene el lexema reconocido más recientemente.
- o yyin: representa el archivo que será analizado.

En esta sección también se definen las constantes que representan los tipos de datos.

- o T_INT
- o T_FLOAT
- o T_STRING
- o T_VOID

También se utilizan las siguientes constantes para diferenciar si una entrada de la tabla de símbolos es una variable o una función:

- o K_VAR
- o K_FUNC

La estructura SymEntry representa cada elemento de la tabla de símbolos. En ella se almacenan:

- El nombre del identificador.
- Su tipo de dato.
- Su clase (variable o función).
- Si es un arreglo.
- El ámbito al que pertenece.
- La línea donde se declaró.

De igual forma, se incluye el arreglo Syntab, los contadores de símbolos y funciones, el ámbito actual y los contadores para errores léxicos, sintácticos y semánticos.

Las funciones auxiliares que están en esta sección son las siguientes:

- **typeName()**: convierte un código de tipo en un nombre legible.
- **semError()**: muestra un error semántico y también incrementa su contador.
- **lookup()**: busca un identificador visible desde el ámbito actual.
- **lookupSameScope()**: busca un identificador dentro del mismo ámbito.
- **declareVar()**: registra una variable o parámetro en la tabla.
- **startFunction()**: registra una función y abre su ámbito local.
- **endFunction()**: cierra el ámbito local de una función y regresa al global.
- **useVar()**: verifica que una variable sí haya sido declarada antes de usarse.
- **useFunc()**: verifica que una función haya sido declarada antes de llamarse.
- **initSyntab()**: inicializa la tabla de símbolos.
- **printSyntab()**: imprime el contenido de la tabla.

Fuera del bloque `%{ %}` se encuentran las declaraciones de Yacc.

La declaración `%union` define los tipos de datos que pueden transportar los símbolos. El campo `str` se usa para los identificadores y para las cadenas, `ival` para números enteros y `dval` para números reales.

```
%union {  
    char *str;  
    int ival;  
    double dval;  
}
```

Los símbolos terminales se declaran por medio de %token. Los tokens que transportan un valor indican el campo de %union que utilizan.

```
%token <str> ID STRING_LITERAL
%token <ival> INTEGER
%token <dval> REAL
```

La declaración de %type indica el tipo de valor producido por los no terminales. El type_specifier se encarga de devolver el código interno de un tipo de dato, mientras que var por otro lado devuelve el nombre del identificador reconocido.

```
%type <ival> type_specifier
%type <str> var
```

Las siguientes declaraciones se usan para resolver la ambigüedad del else. El token LOWER_THAN_ELSE tiene una menor precedencia que el ELSE y eso permite que cada else se relacione con el if más cercano.

```
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
```

Por último, el %start program establece a program como el símbolo inicial de la gramática

4.2. Sección de reglas gramaticales

Esta sección tiene las producciones de la gramática final con sus acciones. La estructura general de una regla es la siguiente:

```
lado_izquierdo
: primera_alternativa { acción }
| segunda_alternativa { acción }
;
```

Los valores de los símbolos del lado derecho se consultan con \$1, \$2, etc. El valor que produce el símbolo del lado izquierdo se representa con \$\$.

La regla program establece que un programa está compuesto por una lista de declaraciones. Esta lista puede contener declaraciones de variables o funciones.

La regla type_specifier asigna a cada palabra reservada su código de tipo correspondiente.

```

type_specifier
: INT      { $$ = T_INT; }
| FLOAT    { $$ = T_FLOAT; }
| STRING   { $$ = T_STRING; }
| VOID     { $$ = T_VOID; }
;

```

Las reglas `var_declaration` y `param` se encargan de llamar a `declareVar()` para registrar variables y parámetros en la tabla de símbolos. Esta función también verifica que una variable no sea de tipo void y que no haya sido declarada anteriormente en el presente ámbito.

Después de usar un identificador recibido desde el scanner, su memoria se libera mediante `free()`, debido a que fue reservada previamente con `strdup()`.

La regla `fun_declaration` utiliza una acción intermedia:

```

type_specifier ID LPAREN
{ startFunction($1, $2); free($2); }
params RPAREN compound_stmt
{ endFunction(); }

```

Esta acción se encarga de registrar la función y de abrir su ámbito antes de analizar los parámetros y el cuerpo. Esto también permite que una función recursiva pueda llamarse a sí misma, ya que su nombre se encuentra en la tabla antes de procesar sus instrucciones.

Al terminar el análisis, `endFunction()` regresa al ámbito global.

Las reglas `var` y `call` usan las funciones `useVar()` y `useFunc()`. Estas funciones verifican que las variables y funciones hayan sido declaradas antes de usarse.

La regla de asignación permite asignar una expresión o una cadena literal a una variable:

```

assignment_stmt
: var ASSIGN expression SEMICOLON
| var ASSIGN STRING_LITERAL SEMICOLON
;

```

La estructura que está formada por `expression`, `arithmetic_expression`, `term` y `factor`, es la que establece la precedencia de los operadores. La multiplicación y la división se procesen antes que la suma y la resta. La recursión izquierda se encarga de que las operaciones se agrupen de izquierda a derecha.

La regla del condicional sin else resuelve el problema del dangling else y provoca que un else se relacione con el if más cercano.

```
%prec LOWER_THAN_ELSE
```

También se incluye una recuperación de errores por medio de las siguientes dos producciones

```
declaration
    : error SEMICOLON { yyerrok; }

statement
    : error SEMICOLON { yyerrok; }
```

Cuando se presenta un error, el parser intenta avanzar hasta encontrar un punto y coma. Posteriormente, `yyerrok` restablece el estado del parser para poder continuar con el análisis. Esto permite reportar más de un error que ocurre dentro de un mismo archivo.

La gramática final y la implementación es directa. Cada producción que se presentó en el análisis aparece como una regla dentro de `parser.y`.

4.3. Sección de código de usuario

Esta sección contiene código C válido. Esta sección comienza después del segundo `%%`. En esta implementación contiene las funciones `yyerror()` y `main()`. Estas se explican más detalladamente a continuación:

- o **Función `yyerror()`:** Yacc llama a esta función cuando encuentra un error sintáctico. La función muestra el número de línea en el que ocurrió el error, el mensaje generado por el parser y el lexema cercano al error. Además incrementa el contador `syntaxErrors`. Cuando el error ocurre al final del archivo y `yytext` está vacío, la función usa el mensaje `end of file`.

```
void yyerror(const char *msg) {
    const char *nearToken;

    if (yytext == NULL || yytext[0] == '\0')
        nearToken = "end of file";
    else
        nearToken = yytext;

    fprintf(stderr,
        "Syntax error at line %d: %s near '%s'\n",
        yylineno,
        msg,
        nearToken);

    syntaxErrors++;
}
```

- **main():** Esta función hace los siguientes pasos:
 - Verifica que el usuario sí haya proporcionado el nombre de un archivo.
 - Abre el archivo con `fopen()`.
 - Asigna el archivo a la variable `yyin`.
 - Inicia la tabla de símbolos por medio de `initSymtab()`.
 - Llama a `yyparse()` para empezar el análisis.
 - Revisa los contadores `lexical_errors`, `syntaxErrors`, y `semanticErrors`.
 - Muestra si el análisis terminó correctamente y si no, indica la cantidad de errores encontrados.
 - Imprime la tabla de símbolos por medio de `printSymtab()`.
 - Cierra el archivo.
 - Devuelve un 0 si no se encuentran errores y 1 cuando existe al menos un error.
 - El resultado se considera exitoso solo cuando los contadores tienen un valor de cero.

4.4. Código completo del archivo Yacc

A continuación se presenta el código completo del archivo `parser.y`. Aquí se incluye todo lo explicado en las secciones anteriores.

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Variables y funciones que provee el scanner generado por flex */
extern int  yylex(void);
extern int  yylineno;      /* numero de linea actual (%option yylineno) */
extern char *yytext;       /* lexema actual reconocido por el scanner */
extern FILE *yyin;         /* archivo de entrada del scanner */

void yyerror(const char *msg);

/*--- Codigos de tipo de dato -----*/
#define T_INT      0
```



```

#define T_FLOAT 1
#define T_STRING 2
#define T_VOID 3

/*--- Clase de simbolo -----*/
#define K_VAR 0
#define K_FUNC 1

/*--- Estructura de La Tabla de Simbolos -----*
 * Cada identificador declarado (variable o funcion) genera una entrada. *
 * El campo 'scope' distingue el ambito: 0 = global. n mayor a 0 es ambito *
 * local de la n-esima funcion. Las entradas NO se eliminan al terminar *
 * solo se vuelven invisibles para la busqueda, de modo que la tabla final *
 * impresa muestre TODOS los simbolos declarados durante el analisis. *
 *-----*/
#define MAXSYM 1000
#define MAXNAME 256

typedef struct {
    char name[MAXNAME];
    int type; /* T_INT, T_FLOAT, T_STRING, T_VOID */
    int kind; /* K_VAR o K_FUNC */
    int isArray; /* 1 si es arreglo, 0 si no */
    int scope; /* 0 = global; n = id de la funcion */
    int line; /* Linea de declaracion */
} SymEntry;

SymEntry symtab[MAXSYM];
int symCount = 0;

char funcNames[MAXSYM][MAXNAME]; /* nombre de cada funcion por su id de ambito */
int currentScope = 0; /* 0 = global. Menor a 0, estoy dentro de una
funcion */
int funcCounter = 0; /* asigna un id de ambito unico por funcion */

int syntaxErrors = 0;
int semanticErrors = 0;
int lexical_errors = 0; /* Lo incrementa el scanner (scanner.L) */

/*--- Utilidades de impresion -----*/
const char *typeName(int t) {
    switch (t) {
        case T_INT: return "int";
        case T_FLOAT: return "float";
        case T_STRING: return "string";
        case T_VOID: return "void";
        default: return "?";
    }
}

void semError(const char *msg, const char *name) {
    fprintf(stderr, "Semantic error at line %d: %s '%s'\n", yylineno, msg, name);
    semanticErrors++;
}

/*--- Operaciones sobre La Tabla de Simbolos -----*/

/* Busca un identificador visible desde el ambito actual.
 * Prioriza una declaracion local. Devuelve el indice o -1 si no es visible. */
int lookup(const char *name) {
    int i, found = -1;
    for (i = 0; i < symCount; i++) {
        if (strcmp(symtab[i].name, name) == 0) {
            if (symtab[i].scope == currentScope) return i; /* Local: prioridad */
            if (symtab[i].scope == 0) found = i; /* global: candidato */
        }
    }
    return found;
}

```

```

    }
}
return found;
}

/* Busca una redeclaracion en el mismo ambito actual. */
int lookupSameScope(const char *name) {
    int i;
    for (i = 0; i < symCount; i++)
        if (symtab[i].scope == currentScope && strcmp(symtab[i].name, name) == 0)
            return i;
    return -1;
}

/* Inserta una variable en el ambito actual (global o local). */
void declareVar(int type, const char *name, int isArray) {
    if (type == T_VOID) {
        /* restriccion: no hay variables void */
        semError("'void' is not a valid type for a variable", name);
        return;
    }
    if (lookupSameScope(name) != -1) {
        semError("identifier already declared in this scope", name);
        return;
    }
    if (symCount >= MAXSYM) return;
    strncpy(symtab[symCount].name, name, MAXNAME - 1);
    symtab[symCount].name[MAXNAME - 1] = '\0';
    symtab[symCount].type = type;
    symtab[symCount].kind = K_VAR;
    symtab[symCount].isArray = isArray;
    symtab[symCount].scope = currentScope;
    symtab[symCount].line = yylineno;
    symCount++;
}

/* Registra una funcion en el ambito global y abre un nuevo ambito local. */
void startFunction(int type, const char *name) {
    if (lookupSameScope(name) != -1) { /* currentScope vale 0 aqui (global) */
        semError("function already declared", name);
    } else if (symCount < MAXSYM) {
        strncpy(symtab[symCount].name, name, MAXNAME - 1);
        symtab[symCount].name[MAXNAME - 1] = '\0';
        symtab[symCount].type = type;
        symtab[symCount].kind = K_FUNC;
        symtab[symCount].isArray = 0;
        symtab[symCount].scope = 0; /* Las funciones viven en el global */
        symtab[symCount].line = yylineno;
        symCount++;
    }
    funcCounter++; /* nuevo id de ambito para esta funcion */
    currentScope = funcCounter;
    strncpy(funcNames[currentScope], name, MAXNAME - 1);
    funcNames[currentScope][MAXNAME - 1] = '\0';
}

/* Cierra el ambito de la funcion: se vuelve al ambito global. */
void endFunction(void) {
    currentScope = 0;
}

/* Verifica que una variable este declarada antes de usarse. */
void useVar(const char *name) {
    int i = lookup(name);
    if (i == -1) {
        semError("variable not declared before use", name);
    }
}

```

```

    } else if (symtab[i].kind != K_VAR) {
        semError("identifier is not a variable", name);
    }
}

/* Verifica que una funcion este declarada antes de llamarse. */
void useFunc(const char *name) {
    int i = lookup(name);
    if (i == -1) {
        semError("function not declared before use", name);
    } else if (symtab[i].kind != K_FUNC) {
        semError("identifier is not a function", name);
    }
}

void initSymtab(void) {
    symCount = 0; currentScope = 0; funcCounter = 0;
    strcpy(funcNames[0], "global");
}

void printSymtab(void) {
    int i;
    printf("\nSYMBOL TABLE\n");
    printf("-----\n");
    printf("%-20s %-8s %-9s %-7s %-12s %-5s\n",
           "Name", "Type", "Kind", "Array", "Scope", "Line");
    printf("-----\n");
    if (symCount == 0) { printf("(empty)\n"); return; }
    for (i = 0; i < symCount; i++) {
        printf("%-20s %-8s %-9s %-7s %-12s %-5d\n",
               symtab[i].name,
               typeName(symtab[i].type),
               symtab[i].kind == K_FUNC ? "function" : "variable",
               symtab[i].isArray ? "yes" : "no",
               symtab[i].scope == 0 ? "global" : funcNames[symtab[i].scope],
               symtab[i].line);
    }
}

/* Tipos de valor semantico que los simbolos pueden transportar (%union). */
%union {
    char *str; /* Lexema de ID y STRING_LITERAL */
    int ival; /* valor de INTEGER y codigo de tipo */
    double dval; /* valor de REAL */
}

/* Tokens terminales. Deben coincidir con los que devuelve scanner.L.
 * bison -y -d genera y.tab.h con estas definiciones para el scanner. */
%token <str> ID STRING_LITERAL
%token <ival> INTEGER
%token <dval> REAL

%token INT FLOAT STRING RETURN IF ELSE WHILE READ WRITE VOID
%token PLUS MINUS TIMES DIVIDE
%token LT LTE GT GTE EQ NEQ ASSIGN
%token SEMICOLON COMMA LPAREN RPAREN LBRACKET RBRACKET LBRACE RBRACE

/* Tipos de los no-terminales que devuelven un valor. */
%type <ival> type_specifier
%type <str> var

%nonassoc LOWER_THAN_ELSE

```

```

%nonassoc ELSE

%start program

%%
/*===== SECCION DE REGLAS =====*/

program
    : declaration_list
    ;

declaration_list
    : declaration_list declaration
    | declaration
    ;

declaration
    : var_declaration
    | fun_declaration
    | error SEMICOLON { yyerrok; } /* recuperacion a nivel declaracion */
    ;

var_declaration
    : type_specifier ID SEMICOLON
      { declareVar($1, $2, 0); free($2); }
    | type_specifier ID LBRACKET INTEGER RBRACKET SEMICOLON
      { declareVar($1, $2, 1); free($2); }
    ;

type_specifier
    : INT { $$ = T_INT; }
    | FLOAT { $$ = T_FLOAT; }
    | STRING { $$ = T_STRING; }
    | VOID { $$ = T_VOID; }
    ;

fun_declaration
    : type_specifier ID LPAREN
      { startFunction($1, $2); free($2); } /* accion embebida: abre ambito */
      params RPAREN compound_stmt
      { endFunction(); }
    ;

params
    : param_list
    | VOID
    ;

param_list
    : param_list COMMA param
    | param
    ;

param
    : type_specifier ID { declareVar($1, $2, 0); free($2); }
    | type_specifier ID LBRACKET RBRACKET { declareVar($1, $2, 1); free($2); }
    ;

compound_stmt
    : LBRACE local_declarations statement_list RBRACE
    ;

local_declarations
    : local_declarations var_declaration
    | /* epsilon */

```

```

;

statement_list
: statement_list statement
| /* epsilon */
;

statement
: assignment_stmt
| call_stmt
| compound_stmt
| selection_stmt
| iteration_stmt
| return_stmt
| input_stmt
| output_stmt
| error SEMICOLON { yyerrok; } /* recuperacion a nivel sentencia */
;

assignment_stmt
: var ASSIGN expression SEMICOLON { free($1); }
| var ASSIGN STRING_LITERAL SEMICOLON { free($1); free($3); }
;

call_stmt
: call SEMICOLON
;

selection_stmt
: IF LPAREN expression RPAREN statement %prec LOWER_THAN_ELSE
| IF LPAREN expression RPAREN statement ELSE statement
;

iteration_stmt
: WHILE LPAREN expression RPAREN statement
;

return_stmt
: RETURN SEMICOLON
| RETURN expression SEMICOLON
;

input_stmt
: READ var SEMICOLON { free($2); }
;

output_stmt
: WRITE expression SEMICOLON
;

var
: ID { useVar($1); $$ = $1; }
| ID LBRACKET arithmetic_expression RBRACKET { useVar($1); $$ = $1; }
;

expression
: arithmetic_expression relop arithmetic_expression
| arithmetic_expression
;

relop
: LTE | LT | GT | GTE | EQ | NEQ
;

arithmetic_expression

```

```

        : arithmetic_expression addop term
        | term
        ;

addop
    : PLUS | MINUS
    ;

term
    : term mulop factor
    | factor
    ;

mulop
    : TIMES | DIVIDE
    ;

factor
    : LPAREN arithmetic_expression RPAREN
    | var { free($1); }
    | call
    | INTEGER
    | REAL
    ;

call
    : ID LPAREN args RPAREN { useFunc($1); free($1); }
    ;

args
    : arg_list
    | /* epsilon */
    ;

arg_list
    : arg_list COMMA arithmetic_expression
    | arithmetic_expression
    ;

%%
/*===== SECCION DE CODIGO =====*/

/* yacc llama a yyerror cuando detecta un error sintactico. Reportamos la
 * linea y el lexema cercano para dar contexto. */
void yyerror(const char *msg) {
    const char *nearToken;

    if (yytext == NULL || yytext[0] == '\0')
        nearToken = "end of file";
    else
        nearToken = yytext;

    fprintf(stderr, "Syntax error at line %d: %s near '%s'\n",
            yylineno, msg, nearToken);
    syntaxErrors++;
}

int main(int argc, char *argv[]) {
    setbuf(stdout, NULL);
    if (argc < 2) {
        fprintf(stderr, "Usage: %s source_file.cmm\n", argv[0]);
        return 1;
    }
    yyin = fopen(argv[1], "r");
    if (yyin == NULL) {

```

```
        fprintf(stderr, "Error: could not open file %s\n", argv[1]);
        return 1;
    }

    initSymtab();
    printf("PARSING: %s\n", argv[1]);
    printf("=====\n");

};

yyparse();

printf("\n");
if (lexical_errors == 0 && syntaxErrors == 0 && semanticErrors == 0)
    printf("RESULT: Parsing completed successfully (no lexical, syntax or semantic
errors).\n");
else
    printf("RESULT: %d lexical, %d syntax and %d semantic error(s) found.\n",
        lexical_errors, syntaxErrors, semanticErrors);

printSymtab();

fclose(yyin);
return (lexical_errors || syntaxErrors || semanticErrors) ? 1 : 0;
}
```

5. Verificación, validación y pruebas

5.1. Modelo o estrategia de pruebas

La verificación se encarga de determinar si el sistema cumple con las especificaciones que se pidieron mientras que la validación consiste en comprobar que el sistema sí cumple con el funcionamiento esperado [4].

Para llevar a cabo ambas verificaciones, se diseñaron varios casos de prueba. En cada caso se seleccionó una característica del parser, se preparó un archivo de entrada y se definió el resultado esperado. Finalmente el resultado se comparó con el obtenido durante la ejecución.

Los elementos que se probaron fueron los siguientes:

- Estructura general de un programa.
- Declaraciones de variables y funciones.
- Parámetros.
- Variables locales y globales.
- Arreglos.
- Asignaciones de expresiones y cadenas.
- Condicionales con else anidado.
- Ciclos while.
- Precedencia de operadores aritméticos.
- Llamadas a funciones.
- Detección de errores léxicos.
- Detección y recuperación de errores sintácticos.
- Uso de variables y funciones no declaradas.
- Redeclaración de identificadores.
- Declaración de variables de tipo void.

Se usaron programas válidos y no válidos para comprobar tanto el reconocimiento correcto de la gramática como la detección de errores.

Como verificación adicional, se generó el archivo y.output por medio de make report. Este comando lo ejecuta Bison con la opción -v. Durante la construcción no se reportaron conflictos sin resolver. Esto indica que Bison pudo generar el parser con la gramática implementada y que la ambigüedad del else fue resuelta mediante las declaraciones de precedencia.

5.2. Casos de prueba

Los ocho casos de prueba están en la carpeta test y todos se pueden ejecutar con el comando make test.

Para mostrar solo el resultado general de cada archivo se usa make test-sum. Igual es posible ejecutar un caso individual con el comando ./parser tst/nombre_del_archivo.cmm. La tabla 4 presenta el objetivo y los resultados de cada prueba.

ID	Objetivo	Archivo de entrada	Resultado esperado	Resultado obtenido	Estado
T1	Probar el programa completo de la especificación	test_programa_completo.cmm	Sin errores y con la tabla de símbolos completa	No se reportaron errores y se generó la tabla de símbolos	Cumplido
T2	Probar if/else anidado, while y expresiones	test_valido_simple.cmm	Sin errores	No se reportaron errores	Cumplido
T3	Probar un error sintáctico y su recuperación	test_error_sintactico.cmm	Reportar un error sintáctico y continuar el análisis	Se reportó un error sintáctico	Cumplido
T4	Probar el uso de una variable no declarada	test_no_declarada.cmm	Un error semántico	Se reportó un error semántico	Cumplido
T5	Probar una redeclaración	test_redeclarada_void.cmm	Dos errores semánticos	Se reportaron dos	Cumplido

	y una variable de tipo void			errores semánticos	
T 6	Probar una llamada a una función no declarada	test_funcion_no_declarada.cmm	Un error semántico	Se reportó un error semántico	Cumplido
T 7	Probar expresiones aritméticas sin espacios	test_sin_espacios.cmm	Sin errores	No se reportaron errores	Cumplido
T 8	Probar un símbolo inválido dentro de un programa	test_error_lexico.cmm	Un error léxico y un resultado diferente de éxito	Se reportó un error léxico y el programa no indicó éxito	Cumplido

Tabla 4. Objetivo y resultados de casos de prueba.

5.3. Justificación de las pruebas

- **T1 Programa completo:** Esta prueba usa el programa de ejemplo de las instrucciones. Incluye declaraciones globales, funciones, parámetros, arreglos, variables locales, ciclos, condicionales, llamadas a funciones, asignaciones de cadenas y sentencias de retorno. Se usó como prueba de integración porque permite comprobar en conjunto la mayor parte de las producciones y la construcción de la tabla de símbolos.
- **T2 Condicional anidado y expresiones:** Esta prueba verifica que el else se relacione solo con el if más cercano. También prueba el ciclo while y el uso correcto de operadores aritméticos y relacionales.
- **T3. Error sintáctico y recuperación:** El archivo tiene un punto y coma que falta y un paréntesis sin cerrar. Cuando el parser detecta el primer error, descarta tokens hasta encontrar un ; y después continúa con el análisis. Así, los dos problemas pueden generar un solo mensaje de error.

- **T4. Variable no declarada:** Esta prueba usa la variable `z` sin haberla declarado. La función `useVar()` la busca en la tabla de símbolos y al no encontrarla marca un error semántico.
- **T5. Redeclaración y variable void:** Esta prueba verifica dos errores. Primero, `a` se declara dos veces en el mismo ámbito así que se reporta una redeclaración. Después, se intenta declarar `b` como `void`, aunque ese tipo solo se permite en funciones. Ambas validaciones se realizan en `declareVar()`.
- **T6. Función no declarada:** Esta prueba llama a `helper()` sin haberla declarado antes. La función `useFunc()` busca su nombre en la tabla de símbolos y, como no lo encuentra, reporta un error semántico.
- **T7. Expresiones sin espacios:** Esta prueba usa expresiones sin espacios, como `5-3`, `a+1` y `10*2-4/2`. Se encarga de verificar que `+` y `-` sean reconocidos como operadores y que la multiplicación y la división se resuelvan antes que la suma y la resta.
- **T8. Error léxico:** Esta prueba incluye el símbolo inválido `@`. El scanner reporta el error e incrementa `lexical_errors`. Al finalizar, `main()` revisa este contador y evita mostrar que el análisis terminó correctamente.

5.4. Evidencias de ejecución

```
(base) anavcampos@Anas-MacBook-Pro-5 AnalizadorSintactico % make test
=====
TEST: tests/test_error_lexico.cmm
=====
PARSING: tests/test_error_lexico.cmm
=====
=====
Lexical error at line 3: invalid symbol '@'

RESULT: 1 lexical, 0 syntax and 0 semantic error(s) found.

SYMBOL TABLE
-----
-----
Name                Type      Kind      Array  Scope    Line
-----
a                    int       variable  no     global   3
main                 void      function  no     global   4

=====
TEST: tests/test_error_sintactico.cmm
```

```

=====
PARSING: tests/test_error_sintactico.cmm
=====
=====
Syntax error at line 5: syntax error near 'a'

RESULT: 0 lexical, 1 syntax and 0 semantic error(s) found.

SYMBOL TABLE
-----
-----
Name                Type      Kind      Array  Scope    Line
-----
-----
a                    int       variable  no      global   2
main                 void      function  no      global   3

=====
TEST: tests/test_funcion_no_declarada.cmm
=====
PARSING: tests/test_funcion_no_declarada.cmm
=====
=====
Semantic error at line 3: function not declared before use 'helper'

RESULT: 0 lexical, 0 syntax and 1 semantic error(s) found.

SYMBOL TABLE
-----
-----
Name                Type      Kind      Array  Scope    Line
-----
-----
main                 void      function  no      global   2

=====
TEST: tests/test_no_declarada.cmm
=====
PARSING: tests/test_no_declarada.cmm
=====
=====
Semantic error at line 5: variable not declared before use 'z'

RESULT: 0 lexical, 0 syntax and 1 semantic error(s) found.

SYMBOL TABLE
-----
-----
Name                Type      Kind      Array  Scope    Line
-----
-----

```

```

-----
-----
a          int      variable no      global      2
main       void     function no      global      3

=====
TEST: tests/test_programa_completo.cmm
=====
PARSING: tests/test_programa_completo.cmm
=====
=====

RESULT: Parsing completed successfully (no lexical, syntax or
semantic errors).

SYMBOL TABLE
-----
-----
Name          Type      Kind      Array      Scope      Line
-----
-----
x              int       variable yes       global      3
s              string    variable no        global      4
f1             float     variable no        global      5
f2             float     variable yes       global      6
miniloc        int       function no        global      8
a              float     variable yes       miniloc     8
low            int       variable no        miniloc     8
high           int       variable no        miniloc     8
i              int       variable no        miniloc     9
y              float     variable no        miniloc     9
k              int       variable no        miniloc     9
sort           void      function no        global      23
a              float     variable yes       sort        23
low            int       variable no        sort        23
high           int       variable no        sort        23
i              int       variable no        sort        24
k              int       variable no        sort        24
t              float     variable no        sort        27
readArray      void      function no        global      37
i              int       variable no        readArray   38
writeArray     void      function no        global      52
i              int       variable no        writeArray  53
main           void      function no        global      62

=====
TEST: tests/test_redeclarada_void.cmm
=====
PARSING: tests/test_redeclarada_void.cmm
=====

```

```
=====
=====
Semantic error at line 3: identifier already declared in this scope
'a'
Semantic error at line 4: 'void' is not a valid type for a variable
'b'

RESULT: 0 lexical, 0 syntax and 2 semantic error(s) found.
```

SYMBOL TABLE

Name	Type	Kind	Array	Scope	Line
a	int	variable	no	global	2
main	void	function	no	global	5

```
=====
TEST: tests/test_sin_espacios.cmm
=====
```

PARSING: tests/test_sin_espacios.cmm

```
=====
RESULT: Parsing completed successfully (no lexical, syntax or
semantic errors).
```

SYMBOL TABLE

Name	Type	Kind	Array	Scope	Line
a	int	variable	no	global	2
main	void	function	no	global	3

```
=====
TEST: tests/test_valido_simple.cmm
=====
```

PARSING: tests/test_valido_simple.cmm

```
=====
RESULT: Parsing completed successfully (no lexical, syntax or
semantic errors).
```

SYMBOL TABLE

----- Name	Type	Kind	Array	Scope	Line

a	int	variable	no	global	2
b	int	variable	no	global	3
main	void	function	no	global	5

6. Referencias

- [1] R. Castelló, Notas de Clase, Tema: "Chapter 3 – Syntax Analysis Part I", TC3002B, School of Engineering and Science, ITESM, Chihuahua, Chih., 2026.
- [2] R. Castelló, Notas de Clase, Tema: "Chapter 4 – Syntax Analysis Part II", TC3002B, School of Engineering and Science, ITESM, Chihuahua, Chih., 2026.
- [3] R. Castelló, Notas de Clase, Tema: "Chapter 0 – Introduction to the Yacc Tool", TC3002B, School of Engineering and Science, ITESM, Chihuahua, Chih., 2026.
- [4] Ian Sommerville, *Software Engineering*, 9th ed. (Addison-Wesley, 2011), 426.